

Anwendungsorientierte Softwareentwicklung

Laborblatt 5: Es kommt das Wort `class` vor

Lernziele

- Auslesen der seriellen Schnittstelle mit Python
- Verwendung von Kontrollstrukturen (`if`, `for`, `while`) und einfachen Datentypen
- Schrittweise Einführung in objektorientierte Programmierung (inkl. Motivation, aber ohne Overkill – der Rest kommt später)

Schritt 1: Eigenes Repository klonen und Game Controller aufbauen

Ziel: Da weitermachen, wo wir letztes Mal aufgehört haben – zunächst beim Mikrocontroller-Code.

Baut auf dem Breadboard den Game Controller aus Laborblatt 4 nach, verbindet ihn mit dem Mikrocontroller und schließt den Mikrocontroller per USB an euren Rechner an.

Wie immer:

- Öffnet zwei Terminals (links zum Editieren, rechts zum Ausführen von Kommandos).
- Öffnet zusätzlich einen Webbrowser und ruft euer Git-Repository unter `https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm` auf.

Linkes Terminal (Editieren)

A) Repository klonen und Verzeichnis wechseln

Klonen des Repositories mit:

```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
```

Verzeichnis wechseln:

```
cd asp-ulm/controller
```

B) Editor öffnen

Öffnet die Datei `controller.ino` im Editor eurer Wahl.

C) Denken ;-)

Lest den Code aufmerksam durch und überlegt euch, was beim Drücken des Tasters bzw. beim Drehen des Potentiometers passieren sollte.

Rechtes Terminal (Kommandos ausführen)

A) Verzeichnis wechseln

```
cd asp-ulm/controller
```

B) USB-Port für WSL sichtbar machen

```
usb_check
```

C) Mikrocontroller flashen

```
make upload
```

D) Seriellen Monitor starten

```
make monitor
```

E) Testen

Überprüft, ob beim Drücken des Tasters und Drehen des Potentiometers das erwartete Verhalten eintritt.

Schritt 2: Serieller Monitor, Marke Eigenbau

Ziel: Die serielle Schnittstelle mit einem Python-Programm auslesen.

Das Besondere an unserer Eigenentwicklung: Es wird nicht Zeichen für Zeichen ausgegeben, sondern erst dann, wenn eine vollständige Zeile (also bis zum Zeilenumbruch) empfangen wurde. Sobald das der Fall ist, **wird diese Zeile auf einen Schlag ausgegeben**. Solange das nicht der Fall ist, passiert nichts.

Vorbereitung

1. Linkes Terminal

A) Verzeichnis wechseln

```
cd ../python
```

B) Editor öffnen

Öffnet die Datei `serial-reader.py` im Editor eurer Wahl.

C) Denken ;-)

Lest den Code aufmerksam durch und überlegt euch, was beim Drücken des Tasters bzw. beim Drehen des Potentiometers passieren sollte. Geht ggf. nochmal Schritt 6 aus Laborblatt 4 durch, um euch klarzumachen:

- (i) Was bewirken `ser.in_waiting` und `ser.read`?
- (ii) Was tun die übrigen Anweisungen?
- (iii) Wie sind die Schleifen verschachtelt – und warum?

2. Rechtes Terminal

A) Verzeichnis wechseln

```
cd ../python
```

B) Testen

Startet das Programm und überprüft, ob beim Drücken des Tasters und Drehen am Potentiometer das erwartete Verhalten eintritt.

Ihr solltet nun an dem Punkt sein, dass einzelne Bytes empfangen werden. Zum Beispiel wird `ch` als `b'X'` ausgegeben – ein Objekt vom Typ `bytes`, also ein Datentyp für "nackte" Binärdaten. Wie man `ch` in lesbaren Text umwandelt, erkunden wir gleich experimentell in einer Python-Shell.

Aufgaben

1. Falls nicht bereits geschehen, beendet mit `Control-C` das Python-Programm im rechten Terminal.
2. Startet die interaktive Python-Shell mit `python`
3. Die Python-Shell funktioniert ähnlich wie die Terminal-Shell, mit der ihr durch das Dateisystem navigiert oder Befehle ausführt – nur dass ihr hier Python-Anweisungen eintippt.

Gebt nacheinander Folgendes ein:

- (1) `ch = b'A'`
- (2) `print(ch)`
- (3) `type(ch)`
- (4) `s = bytes.decode(ch)`
- (5) `type(s)`

Ihr solltet beobachten:

```
>>> ch = b'A'
>>> print(ch)
b'A'
>>> type(ch)
<class 'bytes'>
>>> type(ch)
<class 'bytes'>
>>> s = bytes.decode(ch)
>>> type(s)
<class 'str'>
```

Mit `type(...)` kann man den Typ einer Variablen abfragen. Und mit der Funktion `bytes.decode(...)` kann man ein bytes-Objekt wie `ch` in einen String umwandeln.

4. **Tipp für die Verwendung der Python-Shell:** Mit den **Pfeiltasten nach oben und unten** könnt ihr durch die Eingabe-Historie blättern und bereits eingegebene Befehle erneut ausführen oder bearbeiten.
5. Nun erkunden wir experimentell, wie man Strings aneinanderhängen kann. Gebt dazu in der Python-Shell Folgendes ein:
 - (1) `buffer = ""`
 - (2) `print(buffer)`
 - (3) `buffer = buffer + s`
 - (4) `print(buffer)`
 - (5) `buffer += s`
 - (6) `print(buffer)`

Ihr seht:

- Mit `+` kann man zwei Strings aneinanderhängen.
 - Die Schreibweise `buffer += s` ist äquivalent zu `buffer = buffer + s`, aber kürzer und praktischer.
6. Jetzt simulieren wir mit einer for-Schleife, dass wir nacheinander einzelne Bytes empfangen, diese jeweils in Text umwandeln und an einen String anhängen.

Gebt Folgendes ein:

```
buffer = ""
for ch in [b'A', b'\n', b'X', b' ', b'1', b'\n']:
    ch = bytes.decode(ch)
    buffer += ch
print(buffer)
```

Das ist noch nicht ganz das, was wir später brauchen – aber es geht in die richtige Richtung.

Folgendes passiert hier:

- `buffer` ist anfangs ein leerer String.
- In jeder Schleifenrunde wird ein Byte (`ch`) in einen String umgewandelt.
- Dieser String wird an `buffer` angehängt.
- Am Ende wird `buffer` einmal ausgegeben.

Beachtet: Obwohl es nur eine `print`-Anweisung gibt, werden zwei Zeilen angezeigt.

Das liegt daran, dass `buffer` zwei Zeilenumbrüche (`\n`) enthält – und `print(buffer)` gibt den gesamten Text mit enthaltenen Umbrüchen aus.

7. Jetzt soll der Zeilenumbruch (`\n`) nicht mehr an `buffer` angehängt werden. Stattdessen soll in diesem Fall der aktuelle Inhalt von `buffer` ausgegeben und `buffer` anschließend als leerer String zurückgesetzt werden. Ändert dazu den Code wie folgt:

```
buffer = ""
for ch in [b'A', b'\n', b'X', b' ', b'1', b'\n']:
    ch = bytes.decode(ch)
    if ch == '\n':
        print(f"Zeile> {buffer}")
        buffer = ""
    else:
        buffer += ch
```

Mit `print(f"Zeile> {buffer}")` wird eine formatierte Ausgabe erzeugt: Der Platzhalter `{buffer}` wird automatisch durch den aktuellen Inhalt der Variablen `buffer` ersetzt.

```
Zeile> A
Zeile> X 1
```

8. Denn: Der Mikrocontroller sendet Zeilenumbrüche nicht wie in der Unix-Welt üblich als einzelnes `b'\n'`, sondern in einem Format, das auch von der Windows-Welt verstanden wird: Ein `b'\r'` (Wagenrücklauf) gefolgt von `b'\n'` (Zeilenvorschub).

Das ist aber eine prima Gelegenheit, um ein `elif` und das Schlüsselwort `pass` einzuführen – damit wir das Zeichen `\r` einfach ignorieren, wie es sich gehört. Ändert dazu den Code wie folgt:

```
buffer = ""
for ch in [b'A', b'\n', b'X', b' ', b'1', b'\n']:
    ch = bytes.decode(ch)
    if ch == '\n':
        print(f"Zeile> {buffer}")
        buffer = ""
    elif ch == '\r':
        pass
    else:
        buffer += ch
```

Die `elif`-Bedingung erkennt das Wagenrücklauf-Zeichen (`'\r'`) und überspringt es dann mit `pass`. Alles andere wird wie bisher an `buffer` angehängt.

`pass` bedeutet in Python einfach: Tu nichts. Das ist vergleichbar mit einer `nop`-Instruktion („no operation“) in Assembler – ein Platzhalter, der explizit nichts tut, aber syntaktisch dort stehen muss.

9. Jetzt kommt das Finale

Jetzt seid ihr gefragt, das **er-experimentierte Wissen** umzusetzen!

- **Beendet die Python-Shell** mit `Control-D`.
- Modifiziert `serial-reader.py` so, dass die empfangenen Bytes zeilenweise als Text ausgegeben werden. Sobald eine vollständige Zeile empfangen wurde (endet mit `\n`), soll sie komplett ausgegeben werden. Das Wagenrücklauf-Zeichen (`\r`) wird dabei ignoriert – wie zuvor.
- **Testet euer Programm** im rechten Terminal.
- Wenn alles funktioniert:

```
git commit -a -m "serial reader with line buffer"
git push
```

Damit habt ihr euren eigenen serial non-blocking buffered line reader gebaut – Glückwunsch!

Ziel der folgenden Schritte: Code-Organisation ist alles!

In den nächsten Schritten wird der bestehende Code so umorganisiert, dass er modular, wiederverwendbar und leicht erweiterbar wird – etwa als Baustein für spätere Projekte (wie unser Spiel). Dabei führen wir schrittweise in das Konzept der objektorientierten Programmierung (OOP) ein – jeweils mit einem konkreten Beispiel.

Am Ende sollen zwei zentrale Funktionen (bzw. Methoden) zur Verfügung stehen:

- `connect(port)`
Stellt die Verbindung zur seriellen Schnittstelle her und gibt ein offenes Serial-Objekt zurück, das intern gespeichert wird.
- `readline()`
Prüft, ob eine vollständige Zeile empfangen wurde. Gibt die Zeile als String zurück, falls vorhanden, andernfalls `None`. Die empfangenen Bytes werden dabei intern gepuffert und erst dann als Zeile ausgegeben, wenn ein Zeilenumbruch (`\n`) empfangen wurde. Das Wagenrücklauf-Zeichen (`\r`) wird ignoriert.

Die Umstrukturierung erfolgt in mehreren Etappen:

1. Modularisierung mit globalen Variablen:

Die Logik wird in zwei Funktionen ausgelagert, die globale Variablen `ser` und `buffer` verwenden. So entsteht ein erster, leicht verständlicher modularer Aufbau.

2. Überführung in eine Klasse mit statischen Attributen und Methoden:

Im zweiten Schritt wird die bisherige Struktur in eine Klasse überführt. `ser` und `buffer` werden zu statischen Attributen, `connect` und `readline` zu statischen Methoden. Das verändert die Nutzung nicht, bringt aber Ordnung ins Programm.

3. (Optional) Instanzbasierter Aufbau für Mehrfachverwendung:

In einem weiteren Schritt könnte die Klasse so erweitert werden, dass mehrere serielle Verbindungen gleichzeitig möglich sind – über Instanzattribute und Methoden.

Keine Sorge vor Fachbegriffen:

Lasst euch nicht abschrecken von Wörtern wie *statische Attribute*, *Instanzattribute*, *Methoden*, *Module* oder *Klassen*. Wir entwickeln all das Schritt für Schritt anhand konkreter Beispiele – und ihr werdet sehen: Hinter diesen „fancy“ Ausdrücken stecken einfache, gut verständliche Konzepte.

Schritt 3: Funktionen mit gemeinsamen Variablen

Zum Glück haben wir bereits ein funktionierendes Programm, das die serielle Schnittstelle zeilenweise ausliest. Damit haben wir schon eine gute Vorstellung davon, wie die beiden geplanten Funktionen aussehen sollten:

- Die Funktion `connect` stellt mit `ser = serial.Serial(...)` eine Verbindung zum USB-Port her.
- Die Variable `ser` wird anschließend in der Funktion `readline` verwendet, um
 - abzufragen, ob neue Bytes verfügbar sind (`ser.in_waiting > 0`), und
 - bei Bedarf ein Byte zu lesen (`ser.read(1)`).
- Die Funktion `readline` verwendet außerdem eine Variable `buffer`, an die neue Bytes (nach dem Dekodieren) angehängt werden. Dieser Puffer sollte vor dem ersten Aufruf von `readline` als leerer String initialisiert worden sein – das kann man bequem in `connect` erledigen.

Wir brauchen also zwei Variablen – `ser` und `buffer` – die von beiden Funktionen verwendet werden können. Die naheliegende Lösung: **globale Variablen**.

Allerdings gibt es in Python eine kleine Gemeinheit, wenn man globale Variablen **verändern** will. Wir schauen uns das jetzt an einem einfachen Proof-of-Concept an:

Erstellt im linken Terminal im Editor eurer Wahl die Datei `controller.py` mit folgendem Inhalt:

```
1 ser = None
2 buffer = None
3
4 def connect(port):
5
6     ser = f"connect to {port}"
7     buffer = "empty"
8
9 def readline():
10
11     print(f"reading from {ser}")
12     buffer += ", some data"
13     print(f"buffer: {buffer}")
14
15 connect("/dev/ttyUSB0")
16 readline()
```

Mit „Proof-of-Concept“ ist gemeint: Wir testen an einem möglichst einfachen Beispiel, ob zwei Funktionen dieselben Variablen gemeinsam nutzen können. Das Ziel ist **noch nicht** die serielle Schnittstelle – es geht erstmal nur darum, dass `connect` etwas „vorbereitet“, und `readline` darauf zugreift und weitermacht.

Allerdings klappt das so noch **nicht**. Das werden wir gleich schrittweise **analysieren und beheben**, um dann nach und nach die gewünschte Funktionalität einzubauen.

Aufgaben

1. Führt im rechten Terminal das Programm mit `python controller.py` aus und bestätigt, dass es einen Fehler gibt.
2. Die Fehlermeldung sagt (auf mehr oder weniger verständliche Weise), dass in **Zeile 12** auf die Variable `buffer` zugegriffen wird, es diese aber **nicht gibt**. Das ist eigentlich überraschend – wir haben `buffer` doch in **Zeile 2** außerhalb der Funktion angelegt!
Aber: **Python geht standardmäßig davon aus**, dass innerhalb einer Funktion nur **lokale Variablen verändert** werden. Wenn man also `buffer += ...` schreibt, interpretiert Python

das als Versuch, eine lokale Variable `buffer` zu verändern – und diese wurde ja nicht vorher definiert.

Wir müssen Python also **explizit mitteilen**, dass die globale Variable `buffer` gemeint ist. Das geht, indem wir **Zeile 10** ändern zu:

```
10 global buffer
```

Führt das Programm jetzt erneut aus. Bestätigt, dass es zwar immer noch eine Fehlermeldung gibt – **aber eine andere**.

3. **Die neue Fehlermeldung** besagt, dass der Ausdruck `buffer += ...` in **Zeile 12** nicht erlaubt ist – denn `buffer` hat zu diesem Zeitpunkt den Wert `None`. Im Aufruf von `connect` wird in **Zeile 7** zwar eine Variable `buffer` gesetzt, **aber nicht die globale Variable**, die wir in **Zeile 2** angelegt hatten. Stattdessen hat Python einfach eine **lokale Variable** `buffer` in der Funktion `connect` erzeugt – so, wie es das standardmäßig immer tut. Die globale Variable `buffer` bleibt dadurch **unverändert** auf `None`.

Lösung: Wir müssen auch in der Funktion `connect` explizit sagen, dass wir die globale Variable `buffer` verändern wollen. Und ihr wisst ja inzwischen, wie man das macht 😊

4. **Zwar gibt es jetzt keine Fehlermeldung mehr**, aber das Programm tut **nicht das, was es soll**. Die Ausgabe lautet:

```
reading from None
buffer: empty, some data
```

Erwartet wäre aber:

```
reading from connect to /dev/ttyUSB0
buffer: empty, some data
```

In der Funktion `connect` haben wir zwar die globale `buffer` korrekt verändert – aber `ser` wurde ebenfalls nur lokal initialisiert. Dadurch bleibt die globale Variable `ser` auf `None`, und genau das sehen wir in der Ausgabe von `readline`.

Lösung: Auch die Variable `ser` muss in der Funktion `connect` explizit als **global** deklariert werden. Das geht indem wir **Zeile 5** so schreiben:

```
5 global buffer, ser
```

Die größte Gemeinheit an der Sache: In der Funktion `readline` greifen wir in **Zeile 11** ebenfalls auf `ser` zu – ohne dort `global ser` zu schreiben. **Und trotzdem funktioniert es!**

Warum? Weil wir `ser` in `readline` nur **lesen**, aber **nicht verändern**. Python erlaubt den Zugriff auf globale Variablen **ohne global-Deklaration, solange man sie nur liest**. Erst wenn man versucht, sie zu verändern (wie bei `buffer += ...`), braucht man das `global`. Das ist eine fiese, aber wichtige Unterscheidung!

5. Auch wenn es **technisch nicht notwendig** ist, ändert **Zeile 10** wie folgt ab:

```
10 global buffer, ser
```

Für normalsterbliche Menschen ist dann **sofort klar**, dass in dieser Funktion auf **globale Variablen** zugegriffen wird – egal ob lesend oder schreibend. Das macht den Code **verständlich** und schützt vor Fehlern, wenn man später doch noch etwas an `ser` verändert.

6. Fügt die neue Datei nun zum Repository hinzu:

```
git add controller.py
git commit -a -m "proof of concept"
git push
```

Schritt 4: Den Proof-of-Concept ausbauen

Aus unserem Programm `serial-reader.py` können wir jetzt Code in `controller.py` übernehmen, um die **eigentliche Funktionalität** zu bekommen. Das ist mit etwas Tipparbeit verbunden – aber nicht besonders knifflig.

Unten seht ihr, wie der Code **nach einer ersten Iteration** aussehen kann. Vorher ein kurzer Überblick, **was geändert oder ergänzt wurde**:

- **Zeile 1–2:** Die benötigten Module werden importiert:
 - `serial`, um mit der seriellen Schnittstelle zu kommunizieren,
 - `time`, um den Prozess gelegentlich schlafen zu legen.
- **Zeile 7–11:** Die Funktion `connect` stellt nun tatsächlich eine Verbindung zur seriellen Schnittstelle her und initialisiert `ser`.
- **Zeile 13–27:** Die Funktion `readline` liest alle verfügbaren Zeichen, hängt sie an `buffer` an und prüft auf ein Zeilenende:
 - Wird ein **Linefeed** (`\n`) gefunden, gibt sie den Inhalt von `buffer` zurück und setzt `buffer` zurück.
 - Wird ein **Carriage Return** (`\r`) gefunden, passiert nichts.
 - Ansonsten wird das Zeichen zu `buffer` hinzugefügt.
 - Wichtig: Die Funktion gibt **immer** `None` zurück – selbst im Erfolgsfall! (Das werden wir später noch korrigieren.)
- **Zeile 29–33:** Das Hauptprogramm verwendet jetzt `connect` und `readline`:
 - In **Zeile 29** wird die Verbindung hergestellt.
 - Danach wird in einer Endlosschleife `readline` aufgerufen.
 - Wenn `readline` eine Zeile zurückliefert (also nicht `None`), wird sie ausgegeben.

Das sieht schon recht brauchbar aus – aber: So ganz funktioniert es noch nicht. Denn wie oben erwähnt, gibt `readline` immer `None` zurück, auch wenn eigentlich eine Zeile gelesen wurde.

```

1 import serial
2 import time
3
4 ser = None
5 buffer = None
6
7 def connect(port):
8     global ser, buffer
9     ser = serial.Serial(port, 9600)
10    buffer = ""
11    print(f"connected to {port}")
12
13 def readline():
14     global ser, buffer
15
16     while ser.in_waiting > 0:
17         ch = ser.read(1)
18         ch = bytes.decode(ch)
19         if ch == '\n':
20             print(f"Zeile> {buffer}")
21             buffer = ""
22         elif ch == '\r':
23             pass
24         else:
25             buffer += ch
26     return None
27
28 connect("/dev/ttyUSB0")
29 while 1:
30     line = readline()
31     if line is not None:
32         print(f"readline {line}")
33     time.sleep(0.1)

```


Aufgaben

1. Passt `controller.py` entsprechend an. Bestätigt, dass die empfangenen Daten nicht durch das `print` in **Zeile 32** ausgegeben werden (wie eigentlich vorgesehen), sondern durch die Ausgabe in **Zeile 20**.
2. Es ist naheliegend, in **Zeile 20** den Inhalt von `buffer` zurückzugeben, statt ihn einfach nur auszugeben. Ändert also die Zeile zu `return buffer`. Bestätigt, dass sich das Verhalten nun geändert hat, aber noch nicht ganz das ist was wir wollen.
3. **Lösungsidee für das aktuelle Problem**
Wenn wir in `readline()` eine vollständige Zeile erkennen (also beim `\n`), dann:
(I) merken wir uns den aktuellen Wert von `buffer` in einer neuen lokalen Variable, z. B. `line`
(II) setzen `buffer` zurück auf den leeren String `""`
(III) geben dann `line` zurück

Überlegt selbst, wie ihr diese Strategie in Python-Code umsetzen könnt. Passt die entsprechende Stelle in der Funktion `readline()` entsprechend an. Führt das Programm danach aus und überprüft, ob alles wie gewünscht funktioniert

4. **Wenn jetzt alles wie gewünscht funktioniert:**

```
git commit -a -m "connect, readline"
git push
```

Schritt 5: Mehr Struktur – äh, Klasse bitte!

In unserem Programm gehören gewisse Dinge **offensichtlich zusammen**:

- die beiden globalen Variablen `ser` und `buffer`, und
- die Funktionen `connect` und `readline`.

Wir **wissen** das – zumindest **heute**, weil wir hart daran gearbeitet haben, dass diese vier Einzelgänger endlich **als Team** zusammenarbeiten.

Aber: Wenn man den Code und dessen Funktionsweise **noch nicht kennt**, dann sieht man diese Zusammengehörigkeit **nicht sofort**.

Zum Glück bietet Python ein Sprachmittel, mit dem man genau diese Struktur **sichtbar machen** kann. Und ja – dabei fällt das Zauberwort: `class`

Hier ist ein fast selbsterklärendes Beispiel, wie man unseren Code mit einer Klasse **klarer und strukturierter** ausdrücken kann. Und ja – es ist wieder ein **Proof-of-Concept**, den ihr gleich auf den echten Code übertragen werdet:

```
1 class Controller:
2     ser = None
3     buffer = None
4
5     def connect(port):
6         Controller.ser = f"connected to {port}"
7         Controller.buffer = "empty"
8
9     def readline():
10        print(f"reading from {Controller.ser}")
11        Controller.buffer += ", some data"
12        print(f"buffer: {Controller.buffer}")
13
14 Controller.connect("bar")
15 Controller.readline()
```

Aufgaben

1. Schreibt das obige Beispielprogramm in eine Datei mit dem Namen: `controller_test.py` und führt das Programm aus.
2. Ihr solltet folgende Ausgabe erhalten haben:

```
reading from connected to bar  
buffer: empty, some data
```

Und damit kann man **bereits erahnen**, was dieses kleine Programm ausdrücken will:

Die Variablen `ser` und `buffer` sowie die Funktionen `connect` und `readline` sind Teil der Klasse `Controller`.

Man könnte auch sagen:

- Sie sind in der Klasse gekapselt,
- oder gehören zum Namensraum `Controller`.

Die Welt der objektorientierten Programmierung (OOP) ist zwar voll mit schlaunen Begriffen – aber gemeint ist im Grunde ganz einfach:

- Die Variable `ser` in Zeile 2 heißt eigentlich: `Controller.ser`
- Die Variable `buffer` in Zeile 3 heißt: `Controller.buffer`
- Die Funktion `connect` in Zeile 5 heißt vollständig: `Controller.connect`
- Und `readline` in Zeile 9 ist: `Controller.readline`

👉 **Und genau deshalb brauchen wir auch kein `global` mehr!**

Denn durch dieses Sprachmittel (`class`) ist **ganz klar**, auf **welche Variable** sich der Code bezieht – nämlich auf die, die **zur Klasse `Controller` gehört**. Das sorgt für Übersicht, Struktur – und weniger Überraschungen.

So – und jetzt sollte es doch ein Leichtes sein, dieses Prinzip auf `controller.py` zu übertragen. Und da seid ihr wieder gefragt! 😎

3. Wenn alles wie zuvor klappt (nur eben mit Klasse):

```
git commit -a -m "mit class"  
git push
```